

ESTRATÉGIA QUANTITATIVA MULTI-AGENTE

Desafio Quant AI 2026 — Itaú Asset Management

J • E • M • P • O

Journal • Econ • Math&ML • Program • Orquestrador

Documento de estratégia e arquitetura

Versão atualizada — junho de 2026

Índice

1. O sistema JEMPO em uma página
2. Visão geral da arquitetura e fluxo completo
3. Periodização do sistema
4. Agente JOURNAL — coleta e organização (implementado)
5. Agente ECON — análise fundamentalista e score
6. Agente MATH&ML — previsão de retornos
7. Agente ORQUESTRADOR — decisão e gestão de risco
8. Agente PROGRAM — backtest, Monte Carlo e visualizações
9. Anti-lookahead bias — defesa em profundidade
10. Survivorship bias — universo histórico com membership por data
11. Fontes de dados — arquitetura em camadas
12. Métricas e apresentação para a banca
13. Limitações conhecidas e próximos passos

1. O sistema JEMPO em uma página

JEMPO é uma estratégia quantitativa multi-agente event-driven para ações brasileiras (Ibovespa). O sistema detecta oportunidades após eventos de notícias, combinando análise fundamentalista via IA generativa com machine learning quantitativo e gestão de risco explícita.

Cinco agentes especializados trabalham em pipeline: JOURNAL (coleta dados), ECON (avalia notícias via Claude), MATH&ML (prevê retornos com GradientBoosting), ORQUESTADOR (decide e gerencia risco) e PROGRAM (executa backtest e Monte Carlo). A separação de responsabilidades torna cada peça testável, substituível e defensável independentemente.

Por que essa arquitetura

A maioria dos sistemas quantitativos usa uma única abordagem — ou só machine learning, ou só análise fundamentalista. Cada um isolado tem fraqueza estrutural: ML puro encontra padrões mas não entende mecanismo econômico, podendo descobrir correlações espúrias; fundamentalismo puro avalia empresas bem mas não escala para volume de notícias. JEMPO combina os dois — IA generativa para avaliar impacto econômico qualitativo e ML para quantificar e prever retorno — obtendo um sistema mais robusto que qualquer abordagem isolada.

O diferencial perante a banca

- Rigor metodológico em anti-lookahead: timezone-aware completo, corte da B3 às 17h05, decisão na abertura, DT_RECEB da CVM substituindo lag heurístico, corte do IPCA pela data de divulgação (não pela referência), validação defensiva em três camadas.
- Universo histórico com membership por data: empresas que quebraram (AMER3, IRBR3) são incluídas e excluídas conforme sua presença real no IBOV, eliminando survivorship bias.
- Fundamentos via CVM — fonte regulatória oficial, mesma usada por gestoras profissionais — em vez de yfinance.
- Cálculo TTM para itens de fluxo e point-in-time para balanço, padrão de mercado para P/L e múltiplos.
- Arquitetura modular das fontes externas: cada API (CVM, GDELT, NewsAPI) tem classe própria em agents/sources/, com testes isolados e tolerância a falha de uma fonte sem derrubar o pipeline.
- Backtest out-of-sample em 2024-2025 com walk-forward mensal, custos de transação reais e validação por Monte Carlo.

2. Visão geral da arquitetura e fluxo completo

2.1 Fluxo de uma decisão

Em cada dia útil do backtest, para cada ticker do universo ativo naquela data, o sistema percorre o pipeline:

- JOURNAL coleta notícias relevantes dos últimos 7 dias, fundamentos da CVM com DT_RECEB <= data_limite, preços ajustados e brutos do yfinance, dados macro do BCB SGS e retornos dos pares do setor.
- ECON recebe esses dados e, via Claude API, gera um score combinado de impacto entre -1 e +1, expondo também os componentes separados (notícia, saúde financeira, momento setorial, contexto macro).
- MATH&ML recebe o score do ECON e dezenas de features quantitativas e prevê o retorno em excesso ao mercado nos próximos 5 dias úteis.
- ORQUESTADOR aplica regras de entrada (score > 0.3, retorno previsto > 1.5%, volume > 1.5x média, limites de posição) e decide compra, venda ou aguardar.
- PROGRAM registra a decisão, monitora a posição aberta, aplica regras de saída (stop, take, prazo, reversão de score) e contabiliza custos de 0.4% por round-trip.

2.2 Estrutura de pastas do projeto

A organização do repositório reflete a arquitetura multi-agente, com cada fonte externa em seu próprio módulo:

agents/journal.py – implementação do JOURNAL (orquestra as fontes)

agents/sources/noticia.py – dataclass Noticia compartilhada (quebra import circular)

agents/sources/cvm.py – leitor especializado dos dados abertos da CVM

agents/sources/gdelt.py – leitor do GDELT 2.0 Doc API

agents/sources/newsapi.py – leitor do NewsAPI v2 everything

agents/econ.py – agente de avaliação via Claude API (em desenvolvimento)

agents/math_ml.py – modelo preditivo (em desenvolvimento)

agents/orchestrator.py – coordenação e gestão de risco (em desenvolvimento)

agents/program.py – execução de backtest (em desenvolvimento)

backtest/engine.py – motor de backtest

backtest/monte_carlo.py – simulações Monte Carlo

calibration/econ_calibration.py – calibração do prompt do ECON

data/cache/ – cache geral por fonte (TTL 24h)

data/bloomberg/ – CSVs manuais de notícias do Bloomberg

data/cvm/raw/ – ZIPs originais da CVM

data/cvm/processed/ – DataFrames consolidados em parquet

config.py – UNIVERSO_HISTORICO, WHITELIST_FONTES, TICKER_PARA_NOME, periodização

tests/test_journal.py, test_cvm.py, test_gdelt.py, test_newsapi.py – testes contra APIs reais

2.3 Stack técnica

Python 3.12+, pandas e numpy para séries temporais, yfinance para preços, requests para BCB, CVM, GDELT e NewsAPI, scikit-learn (GradientBoosting) para o MATH&ML, Anthropic API para o ECON, pyarrow para cache em parquet, matplotlib para visualizações. Deduplicação fuzzy de notícias via difflib (stdlib), sem dependência externa. Desenvolvimento no macOS via Claude Code (Anthropic). Repositório privado no GitHub.

3. Periodização do sistema

O sistema cobre 7 anos de dados, divididos em quatro fases com propósitos distintos. A periodização foi desenhada para maximizar diversidade de regimes no treino e validar fora da amostra em dados recentes consolidados.

Fase	Período	Propósito
Warmup	2019	Features de momentum usam 252 dias úteis (~1 ano)
Calibração ECON	2020-2021	Prompt engineering iterativo; meta correlação > 0.15
Treino MATH&ML	2020-2023	4 anos cobrindo COVID, recuperação, alta Selic, recuperação 2023
Backtest OOS	2024-2025	Anos completos com dados consolidados na CVM

3.1 Por que esses períodos e não outros

A calibração do ECON usa 2020-2021 porque dois anos são suficientes para estabelecer significância estatística da correlação entre `score_econ` e retorno real, e porque esse período cobre tanto crise (COVID 2020) quanto recuperação (2021) — testa o ECON em regimes opostos.

O treino do MATH&ML usa 2020-2023 — quatro anos completos. GradientBoosting precisa de exposição a regimes variados para generalizar bem. Esses anos incluem crise de liquidez (mar/2020), recuperação rápida (2021), ciclo de alta da Selic e queda de tech (2022), e recuperação parcial (2023). Treinar em janela menor levaria a overfitting em um regime específico.

O backtest usa 2024-2025 porque estamos em junho/2026 e esses são os anos completos mais recentes com dados consolidados na CVM. Avaliar a estratégia em condições recentes de mercado é mais relevante que avaliar em regime histórico distante das condições atuais.

3.2 Walk-forward — aprendizado contínuo durante o backtest

Durante o backtest, o modelo MATH&ML retreina mensalmente, incorporando o mês recém-realizado ao conjunto de treino. Isso resolve dois problemas simultaneamente: garante que o modelo nunca usa dados futuros (anti-lookahead estrutural) e permite que ele se adapte gradualmente a mudanças no regime de mercado. Em janeiro/2024, o modelo decide baseado em treino 2020-2023. Em fevereiro/2024, incorpora janeiro/2024 real e retreina. E assim segue, mês a mês, até o fim de 2025.

4. Agente JOURNAL — coleta e organização (implementado)

O JOURNAL é o agente de coleta e o único concluído até o momento (junho/2026). Mais de 70 testes automatizados rodam contra APIs reais e passam. É o provedor de dados puro do sistema: coleta, limpa, organiza e entrega dados brutos aos demais agentes. Não avalia, não pondera, não toma decisão alguma — apenas garante que os dados que saem dele são corretos, completos quando possível, e explícitos sobre lacunas e qualidade.

4.1 Arquitetura modular das fontes

Cada fonte externa é uma classe própria em `agents/sources/`: `CVMSource` (`cvm.py`), `GDELTSource` (`gdelt.py`), `NewsAPISource` (`newsapi.py`). A dataclass `Noticia` fica em `noticia.py` — módulo neutro que quebra o import circular entre `journal.py` e as fontes de notícia.

Vantagens dessa separação: cada fonte é testável isolada (`test_cvm.py`, `test_gdelt.py`, `test_newsapi.py`), substituível (trocar fonte mexe em um arquivo só), e falha em uma não vaza para o resto (se GDELT cair, Bloomberg manual + NewsAPI continuam funcionando). O `JournalAgent` apenas orquestra as fontes — o parsing pesado mora nas classes específicas.

4.2 Métodos públicos

- `get_noticias(query, data_limite, lookback_days=7)` → lista de objetos `Noticia` tipados, consultando Bloomberg CSV + GDELT + NewsAPI em cascata, filtrados por `whitelist`, deduplicados, ordenados por peso e recência.
- `get_precos(ticker, data_inicio, data_limite)` → `DataFrame` com OHLCV, duas versões de preço (ajustada `split+dividendo` e bruta) e flags de qualidade por linha.
- `get_fundamentals(ticker, data_limite)` → dataclass `Fundamentals` com último demonstrativo cujo `DT_RECEB <= data_limite`, métricas em TTM ou point-in-time conforme natureza, P/L e P/VP best-effort blindado.
- `get_macro(data_limite)` → dicionário com Selic diária, Selic meta, PTAX e IPCA (cortado pela data de divulgação, não pela referência), todos limitados em `data_limite`.
- `get_retorno_ibovespa(data_inicio, data_limite)` → série de retornos diários do `^BVSP` para cálculo de excesso ao mercado.
- `get_setor(ticker)` → setor padronizado em vocabulário fixo interno.
- `get_retornos_setor(setor, data_limite, janela=60)` → agregação bruta dos retornos dos pares do setor.
- `health_check()` → dict com status de cada fonte (`online`, `offline`, `sem_chave`, `vazio`) para diagnóstico antes de backtest longo.

4.3 Coleta de notícias em camadas

`get_noticias` consulta três fontes em cascata, com critérios complementares: Bloomberg CSV manual (peso 1.0, máxima qualidade editorial, ~80-100 eventos críticos curados manualmente), GDELT (cobertura histórica desde 2015, sem chave, pouco corpo de artigo) e NewsAPI (apenas últimos 30 dias no plano free, mas com corpo do artigo). Nenhuma sozinha basta — combinadas, cobrem qualidade, volume histórico e profundidade recente.

Uma `whitelist` rígida de domínios elimina ruído na origem: `bloomberg.com`, `reuters.com`, `valor.globo.com`, `valor.com.br`, `broadcast.com.br`, `estadao.com.br`, `infomoney.com.br`. Pesos

refletem credibilidade editorial: Bloomberg 1.00, Reuters/Valor 0.95, Broadcast 0.90, Estadão 0.85, InfoMoney 0.75. Qualquer fonte fora da whitelist é descartada antes mesmo de virar objeto Notícia.

Cada fonte tem cache próprio em pickle no diretório data/cache/, com TTL de 24h e chave baseada em hash de query + data_limite. A segunda chamada idêntica é instantânea, o que é crítico para backtests de meses ou anos.

4.4 Deduplicação fuzzy entre fontes

A mesma notícia frequentemente aparece em múltiplas fontes — Bloomberg publica, GDELT indexa, NewsAPI também captura. Antes, a deduplicação era por match exato de título normalizado, mas isso falhava em variações pequenas ("Petrobras anuncia dividendo recorde" vs "Petrobras anuncia dividendo recorde hoje" escapavam como notícias distintas).

A solução implementada usa similaridade de títulos via `difflib.SequenceMatcher` (stdlib, sem dependência externa). Duas notícias são consideradas duplicatas se `ratio > 0.85` E diferença de `timestamp < 24h`. As notícias são processadas em ordem decrescente de peso, então a primeira de cada grupo é sempre a da fonte mais confiável; duplicatas de menor peso são descartadas. Os limiares estão em constantes `_DEDUP_SIM_MIN` (0.85) e `_DEDUP_HORAS_MAX` (24) no `journal.py`.

4.5 Fundamentos via CVM — substituindo o yfinance

O plano original do JEMPO usava `yfinance.Ticker.info` para fundamentos — abordagem comum mas com dois problemas graves. Primeiro, `.info` retorna snapshot atual da empresa, não histórico — usar isso num backtest de 2024 seria dizer que em março de 2024 a empresa tinha os fundamentos de 2026. Lookahead massivo e silencioso. Segundo, mesmo `yfinance.quarterly_financials` cobre apenas os 4-5 trimestres mais recentes, insuficiente para um backtest histórico.

Migramos para os dados abertos da CVM (Comissão de Valores Mobiliários). A CVM publica todos os ITRs (Informes Trimestrais) e DFPs (Demonstrações Financeiras Padronizadas) em formato aberto desde 2011 em `dados.cvm.gov.br`. É a fonte regulatória oficial — a mesma usada por Bloomberg, Refinitiv e gestoras profissionais.

A implementação está em `agents/sources/cvm.py`, módulo separado para não inflar o `journal.py`. O parser trata as peculiaridades da CVM: encoding latin-1 (não UTF-8), separador ponto-e-vírgula, decimal vírgula, valores em milhares de reais, versões reapresentadas filtradas pela máxima, estrutura BPB diferenciada para bancos.

DT_RECEB — anti-lookahead preciso por documento

Cada documento da CVM traz um campo `DT_RECEB` com a data exata em que a CVM o recebeu — ou seja, em que ele virou público. Antes da migração, usávamos um lag heurístico de 45 dias (prazo regulatório máximo). Com `DT_RECEB`, em vez de assumir 45 dias para todos, sabemos por documento a data exata de disponibilidade pública. Empresas frequentemente divulgam antes do prazo; o lag heurístico descartava sinal válido. Argumento robusto para a banca: usamos a data efetiva publicada pela própria CVM, não estimativa.

TTM para fluxo, point-in-time para balanço

A CVM publica DRE acumulado: ITR Q1 é janeiro-março (3 meses), Q2 é janeiro-junho (6 meses), Q3 é 9 meses, DFP é o ano inteiro. Usar valores brutos quebraria comparabilidade entre datas. A solução padrão de mercado é o TTM (Trailing Twelve Months) — 12 meses contínuos, eliminando sazonalidade e tornando todas as datas comparáveis.

A fórmula aproveita uma coluna que a CVM já fornece em todo ITR: `PENÚLTIMO` (mesmo período do ano anterior). Quando o documento mais recente é um ITR (Q1, Q2 ou Q3), o TTM é calculado como

ÚLTIMO + (DFP_ano_anterior - PENÚLTIMO). Quando é uma DFP, o TTM é trivialmente o valor anual. Itens de balanço (ativo, patrimônio, dívida, caixa) ficam point-in-time — patrimônio em 31/março é o valor naquele dia, não acumulado. O dicionário retornado inclui um campo "periodicidade" explícito, mapeando cada métrica para "TTM" ou "point_in_time" — ECON e MATH&ML consultam antes de cálculos derivados.

4.6 Tratamento de qualidade nos preços

Para cada linha do DataFrame de preços, o JOURNAL atribui uma flag de qualidade: volume_zero (dia sem negócio efetivo), outlier_retorno (variação diária maior que 30%, sugerindo split não ajustado ou tick errado), nan (Close ausente) ou ok. O sistema marca o problema em vez de apagar a linha — apagar dado silenciosamente esconde problemas sistemáticos. Marcando, o consumidor decide o que fazer e o problema permanece visível para auditoria.

Forward-fill é opcional, limitado a 3 dias, e aplicado apenas ao Close. Volume nunca é preenchido — inventar volume seria fraude de dado.

5. Agente ECON — análise fundamentalista e score

O ECON é o agente que avalia o impacto econômico de notícias usando IA generativa (Claude via API). Recebe do JOURNAL os dados brutos (notícia + fundamentos + retornos dos pares + macro) e produz um score numérico de impacto entre -1 e +1, junto com os componentes desagregados para uso pelo MATH&ML.

5.1 Por que IA generativa em vez de análise de sentimento tradicional

Análise de sentimento clássica (positivo/neutro/negativo no texto) é frágil. Uma notícia com tom positivo pode ser ruim para a ação: "empresa anuncia aquisição cara" é positivo na superfície mas destrói valor. O ECON usa o Claude para avaliar o mecanismo econômico subjacente — não apenas o tom, mas o canal pelo qual a notícia afeta o caixa, a margem, a posição competitiva ou o múltiplo da empresa.

O Claude é instruído via system prompt a se comportar como analista fundamentalista sênior brasileiro, considerando solidez financeira da empresa, momento setorial e contexto macro ao avaliar cada notícia. A resposta é estruturada via tool use (function calling) — schema rígido em JSON, sem parsing frágil de texto livre.

5.2 Decisão de design: meio-termo entre ECON-decide-tudo e ECON-só-notícia

Houve uma decisão metodológica relevante sobre quanto o ECON deve ponderar. Duas visões competiram:

Visão A: o ECON age como analista completo, ponderando notícia + saúde financeira + momento setorial + macro e produzindo um único score combinado. Mais fiel ao conceito de "analista fundamentalista", mais defensável conceitualmente. Captura interações sutis que modelos lineares não pegam (ex: "notícia ruim, mas empresa tem caixa enorme").

Visão B: o ECON avalia apenas a notícia, e os outros componentes viram features independentes para o MATH&ML aprender os pesos sozinho. Resposta mais forte à pergunta "por que esses pesos?" — porque o ML otimizou, não porque foram arbitrados.

Adotamos um meio-termo: o ECON pondera tudo e produz o score_econ combinado (cumpre Visão A), MAS também expõe os componentes separados como campos individuais (cumpre Visão B). O MATH&ML decide se usa o score final, os componentes separados, ou ambos — testa via validação cruzada qual configuração gera melhor IC. Custo: o prompt do ECON fica um pouco maior. Benefício: mantém o melhor dos dois lados.

5.3 Calibração do ECON (2020-2021)

Antes de usar o ECON em produção, validamos que ele realmente gera scores correlacionados com retornos reais. A calibração funciona aplicando o ECON em notícias históricas de 2020-2021 e comparando o score gerado com o retorno real ajustado ao mercado (excesso sobre Ibovespa) nos 5 dias úteis seguintes. A meta é correlação de Spearman acima de 0.15 — modesta em termos absolutos mas relevante em finanças, onde mercados eficientes deixam pouca margem para sinal.

Se a correlação ficar abaixo da meta, fazemos prompt engineering iterativo: identificamos padrões nos casos em que o ECON mais erra, e ajustamos o prompt para corrigir essas confusões. Ex: se o ECON superestima impacto de notícias políticas sem efeito nos fundamentos, adicionamos instrução para descontar relevância nesses casos.

Importante: limitamos os ciclos de ajuste para evitar overfitting do prompt ao período de calibração. Após no máximo ~10 iterações, congelamos o prompt e medimos a correlação final como métrica de qualidade reportada à banca.

6. Agente MATH&ML — previsão de retornos

O MATH&ML é o agente preditivo. Recebe dezenas de features (incluindo o score do ECON) e prevê o retorno em excesso ao mercado ajustado por beta setorial nos próximos 5 dias úteis. A saída é um número, não uma classe — regressão, não classificação.

6.1 Escolha do modelo: GradientBoosting

Avaliamos três famílias de modelos antes de decidir. LSTM (redes neurais recorrentes) foi descartada porque exigem volume de dados muito maior que temos disponível — com poucos milhares de amostras de treino, redes profundas overfittam dramaticamente. Random Forest foi descartada por ser inferior ao GradientBoosting em problemas com forte componente sequencial e features correlacionadas, como retornos financeiros.

GradientBoosting (scikit-learn implementation) com `max_depth=3` e `learning_rate=0.05` entrega regularização explícita via árvores rasas e taxa de aprendizado conservadora. Cada árvore individualmente captura pouco — o conjunto captura interações sem decorar exemplos específicos. Em finanças, onde a relação sinal/ruído é baixa, essa contenção é essencial para o modelo generalizar.

6.2 Target: retorno em excesso ajustado por beta setorial

O target não é o retorno absoluto da ação, e sim o retorno em excesso ao mercado ajustado pelo beta setorial. Razão: o que importa para a estratégia é capturar sinal específico da empresa, não retorno do mercado que qualquer índice passivo entrega. Se PETR4 sobe 3% num dia em que o Ibovespa subiu 2.5% e o beta setorial é 1.1, o retorno em excesso ajustado é aproximadamente $3\% - 1.1 \times 2.5\% = 0.25\%$ — bem menos impressionante que o número bruto.

Esse ajuste é especialmente importante no período de treino, que inclui o crash COVID de 2020 e a recuperação de 2021. Sem ajuste, o modelo aprenderia a prever direção do mercado em vez de stock-picking. Com ajuste, ele foca no que de fato distingue um sinal de notícia de uma maré de mercado.

6.3 Features do modelo

As features se dividem em quatro grupos, totalizando aproximadamente 30 variáveis por exemplo de treino:

- Score do ECON e componentes desagregados (notícia, saúde financeira, momento setorial, macro).
- Fundamentais TTM da CVM: P/L (best-effort), margem líquida, ROE anualizado, dívida líquida sobre EBITDA, crescimento de receita TTM ano-sobre-ano.
- Momentum técnico: retornos acumulados de 5, 21, 63 e 252 dias úteis, volume relativo (versus média 20d), volatilidade realizada 21d.
- Contexto macro e mercado: Selic atual, variação semanal do dólar, IPCA acumulado 12m, retorno do Ibovespa 5d.

6.4 Métrica de avaliação: IC (Information Coefficient)

Em vez de RMSE ou MAE, usamos o IC — correlação de Spearman entre previsões e retornos realizados. Razão: para uma estratégia de seleção (entra na ação se sinal forte), o que importa é o ordenamento, não o valor absoluto da previsão. Se o modelo classifica corretamente as 5 ações com

maior retorno futuro, mesmo errando os valores absolutos, ele é útil para a estratégia. IC é a métrica padrão da indústria para essa exata situação — falar em IC para a banca é falar a língua de quem avalia fundos quantitativos.

6.5 Validação por TimeSeriesSplit + walk-forward

Durante o desenvolvimento, usamos TimeSeriesSplit do scikit-learn (5 folds sequenciais) sobre o conjunto de treino 2020-2023. Cada fold treina em janela passada e valida no próximo trimestre, sem contaminação temporal. Em produção (backtest 2024-2025), walk-forward mensal: a cada virada de mês, o modelo retreina incorporando o mês anterior real. Em janeiro/2024 decide com modelo treinado em 2020-2023; em fevereiro, retreina com 2020-jan/2024; e segue até dezembro/2025.

7. Agente ORQUESTADOR — decisão e gestão de risco

O ORQUESTADOR é o agente central: coordena o pipeline, consolida as saídas de ECON e MATH&ML, aplica regras de gestão de risco e toma a decisão final de comprar, manter ou vender. É o gestor do fundo na metáfora — ouve analistas e quants, mas decide com cabeça fria sobre risco total.

7.1 Regras de entrada (todas devem ser satisfeitas)

- Score do ECON acima de 0.3 — limiar de impacto relevante.
- Retorno previsto pelo MATH&ML acima de 1.5% em 5 dias úteis.
- Volume do dia acima de 1.5 vezes a média móvel de 20 dias — confirmação de interesse de mercado.
- Menos de 3 posições abertas simultaneamente em todo o portfólio.
- Menos de 2 posições no mesmo setor — diversificação setorial mínima.
- Capital livre disponível para o dimensionamento da posição.

Quando todas as condições são satisfeitas, o ORQUESTADOR registra a decisão de compra. A execução acontece na abertura do próximo pregão (D+1), nunca no fechamento de D — coerente com a regra estrutural de que decisões tomadas às 10h só podem usar dados até o fechamento de D-1.

7.2 Regras de saída (o primeiro critério atingido dispara)

- Stop loss em -8% do preço de entrada — corta perdas antes de virarem catastróficas.
- Take profit em +15% do preço de entrada — realiza ganhos sem ficar esperando reversão.
- Prazo máximo de 5 dias úteis — sai independente do resultado para limitar exposição temporal.
- Reversão de score: se o ECON gerar novo score abaixo de -0.3 sobre a empresa, sai mesmo no lucro.

Regras de saída são o que separa uma estratégia real de um modelo que nunca realiza prejuízo. Sem stop loss, o backtest pode acumular perdas teóricas infinitas em ações que não recuperam. O conjunto de saídas é intencionalmente conservador — preferimos sair cedo demais do que tarde demais.

7.3 Dimensionamento de posição (sizing)

O ORQUESTADOR não aloca o mesmo valor em toda posição — dimensiona conforme a força do sinal:

Condição	% do capital alocado
Score > 0.7 e retorno previsto > 3%	20%
Score > 0.5 e retorno previsto > 2%	15%
Demais casos (sinal moderado)	10%

O dimensionamento é calculado sobre o capital inicial, não o capital corrente — convenção comum no mercado que evita que perdas iniciais reduzam o tamanho das posições subsequentes (e vice-versa), facilitando interpretação e replicação.

7.4 Drawdown protection

Se o drawdown acumulado da estratégia (queda do pico) ultrapassar 15%, o ORQUESTRADOR pausa novas entradas automaticamente. Permanece em modo somente saída até o drawdown recuar a um nível aceitável. Mecanismo de proteção contra deterioração sistêmica que pode indicar que o modelo descalibrou ou que o regime de mercado mudou de forma a invalidar as premissas.

8. Agente PROGRAM — backtest, Monte Carlo e visualizações

O PROGRAM executa o backtest histórico, valida robustez por Monte Carlo e gera os relatórios para a apresentação. É o agente de simulação e análise final.

8.1 Motor de backtest

O backtest itera dia útil a dia útil sobre o período 2024-2025. Em cada dia: o ORQUESTRADOR é consultado para cada ticker do universo ativo naquela data (via `tickers_ativos`), decisões de entrada são registradas para execução na abertura do próximo pregão, posições abertas são monitoradas contra critérios de saída intradia (usando High/Low do candle para detectar se stop ou take foram tocados), e o capital é atualizado.

8.2 Custos de transação realistas

Cada operação (entrada ou saída) incorre em 0.4% de custo, dividido em 0.3% de corretagem efetiva + 0.1% de slippage. Esses valores são coerentes com o mercado brasileiro para tickets pequenos a médios em ações líquidas. Round-trip completo (entrada + saída) custa 0.8%. Estratégias que ignoram custo subestimam dramaticamente o custo real de uma estratégia de alta rotatividade — incluir custos explicitamente é o que separa backtest defensável de backtest de papel.

8.3 Validação por Monte Carlo

Para responder a pergunta inevitável da banca — "o resultado não foi sorte da sequência específica de eventos?" — o PROGRAM roda 10.000 simulações de Monte Carlo. A técnica: pega-se a lista de retornos das operações realizadas no backtest e sorteia aleatoriamente sequências com reposição. Para cada sequência sorteada, calcula-se a evolução do capital. Resultado: distribuição empírica de cenários possíveis a partir das mesmas operações.

Reportam-se três estatísticas: mediana da distribuição (resultado típico), percentis 5 e 95 (pior e melhor 5% dos cenários), e probabilidade de retorno positivo. Se em 90%+ dos cenários a estratégia foi lucrativa, o resultado não é produto de uma sequência específica improvável — é robusto. Esse é o argumento contra crítica de luck.

8.4 Visualizações para a apresentação

O PROGRAM gera automaticamente: curva de capital ao longo do tempo (estratégia versus Ibovespa), distribuição de retornos por operação, drawdown histórico, contribuição por setor, fan chart das simulações Monte Carlo (visualizando incerteza), matriz de operações coloridas por motivo de saída. Tudo em matplotlib para reprodutibilidade e exportação para os slides.

9. Anti-lookahead bias — defesa em profundidade

Lookahead bias é o erro mais grave em backtest. Significa usar no presente uma informação que só estaria disponível no futuro. Quando isso acontece, o backtest produz resultados artificialmente bons que jamais se reproduziriam em operação real. A primeira coisa que uma banca de gestora pergunta a uma estratégia quantitativa é justamente como o sistema garante ausência de lookahead. Por isso a defesa precisa ser visível, rigorosa e a três camadas.

9.1 Camada 1 — Parâmetro `data_limite` obrigatório

Toda função do JOURNAL recebe `data_limite` como parâmetro obrigatório, sempre `timezone-aware` no fuso `America/Sao_Paulo`. Não existe nenhuma função pública que retorne "o dado mais recente disponível" — sempre é "o dado disponível até esta data". Isso elimina por design a possibilidade de algum agente consultar o JOURNAL sem especificar o cutoff temporal.

9.2 Camada 2 — Corte na fonte

Cada fonte de dados é consultada com seu próprio mecanismo de corte. No `yfinance`, o parâmetro `end` é exclusivo, então somamos 1 dia e cortamos pelo `timestamp`; no `BCB SGS`, passamos `dataFinal` na requisição; no `GDELT`, passamos `enddatetime` convertido para UTC; na `CVM`, filtramos por `DT_RECEB` menor ou igual à `data_limite`. O importante é que o corte aconteça na consulta, não depois, evitando trazer dados desnecessários para a memória do processo.

9.3 Camada 3 — Validação defensiva `_assert_no_lookahead`

Antes de retornar qualquer `DataFrame` ou `Series` ao chamador, uma função utilitária verifica que o `timestamp` máximo do índice não ultrapassa `data_limite`. Se ultrapassar, levanta uma exceção `LookaheadError` com contexto (qual método e qual ticker), abortando a execução imediatamente. Essa terceira camada protege contra dois cenários: bugs intermitentes de fontes externas (`yfinance` ocasionalmente devolve 1 dia a mais por causa de `timezone`) e erros futuros nossos em refatorações que esqueçam de aplicar o corte.

9.4 O corte da B3 às 17h05

Decisão estruturante específica do mercado brasileiro: o pregão da B3 termina com call de fechamento, que se encerra às 17h05. Antes desse horário, o candle do dia ainda não fechou, então o último fechamento disponível é o de D-1. O JOURNAL trata isso via função `_ultimo_fechamento_disponivel(agora_aware)` que retorna o candle correto baseado no horário do `timestamp` recebido.

Como o ORQUESTRADOR decide às 10h da manhã (e como 10h é menor que 17h05), por construção, todas as decisões usam dados até o fechamento do dia anterior. Não há janela onde o sistema possa "olhar" o fechamento do próprio dia da decisão. Isso parece detalhe, mas em backtests com milhares de decisões, lookahead de horas se acumula em retornos espúrios.

9.5 `DT_RECEB` para fundamentos — substituindo o lag heurístico

Antes, usávamos um lag heurístico fixo de 45 dias para fundamentos (prazo regulatório máximo da CVM Resolução 80). Após a migração para CVM como fonte primária, descobrimos o campo `DT_RECEB` — data exata em que cada documento foi recebido pela CVM. Substituímos a heurística pelo dado real. Mais preciso (empresas frequentemente divulgam antes do prazo) e mais defensável (data oficial publicada pela própria CVM, não suposição).

9.6 IPCA cortado pela data de divulgação, não pela referência

Detalhe sutil mas importante: o BCB SGS indexa o IPCA pela data de referência (1º dia do mês), mas o índice só é divulgado pelo IBGE cerca de 11 dias após o fim do mês de referência. O IPCA de maio sai por volta de 11 de junho. Cortar pela data de referência (`index <= data_limite`) vazaria inflação que ainda não era pública — um lookahead sutil que pode passar despercebido.

O `get_macro` corta o IPCA pela data de disponibilidade estimada (fim do mês + 11 dias, constante `_LAG_IPCA_DIAS`), não pela referência. As séries diárias (Selic, PTAX) continuam cortadas por dia, o que é adequado dado que o ORQUESTRADOR decide na abertura usando o fechamento de D-1.

10. Survivorship bias — universo histórico com membership por data

Survivorship bias é o segundo erro mais comum em backtests amadores. Acontece quando o universo de ativos considerado contém apenas as empresas que sobreviveram até hoje, excluindo silenciosamente aquelas que quebraram, faliram ou foram excluídas do índice ao longo do tempo. O efeito: o backtest mostra retornos artificialmente positivos porque, por construção, evita as piores ações do período.

10.1 A solução: UNIVERSO_HISTORICO com entrada e saída por ticker

Em vez de uma lista estática de tickers, o JEMPO usa um dicionário `UNIVERSO_HISTORICO` em `config.py` onde cada ticker tem campos de entrada e saída do IBOV. A função `tickers_ativos(data_aware)` retorna apenas os tickers cuja janela [entrada, saída] contém aquela data. Toda iteração sobre o universo — no treino, no backtest, na agregação setorial — usa essa função. Nunca uma lista estática.

Cada entrada do dicionário carrega: o setor padronizado, a data de entrada (None se já estava no IBOV em 01/01/2019), a data de saída (None se ainda está em 31/12/2025), o nível de confiança da informação (alta, média ou baixa), a fonte (carteira oficial da B3, fato relevante, etc.) e o `cd_cvm` para lookup de fundamentos.

10.2 Casos emblemáticos no universo

Incluímos deliberadamente alguns casos emblemáticos de quebra para que o modelo treine vendo o pior cenário:

- AMER3 (Americanas) — saída em 12/jan/2023. A empresa anunciou "inconsistências contábeis" de R\$ 20 bilhões em 11/janeiro, pediu recuperação extrajudicial no dia 12 e a ação despencou 80%+ em dias. Com o backtest em 2024-2025, esse evento cai no período de treino. Argumento poderoso: o modelo aprendeu vendo uma quebra real de empresa do IBOV — não está sendo treinado apenas com sobreviventes.
- IRBR3 (IRB Brasil RE) — saída após escândalo contábil em 2020. Outro caso emblemático também no período de treino. Diversidade de tipos de quebra: a Americanas foi fraude descoberta tarde; o IRB foi crise reputacional rápida.

10.3 Período coberto: 2019-2025

O universo cobre 7 anos para acompanhar todas as fases do sistema. 2019 é warmup (features de momentum precisam de 252 dias úteis prévios). 2020-2021 é calibração do ECON. 2020-2023 é treino do MATH&ML. 2024-2025 é backtest. Em qualquer dessas datas, `tickers_ativos` retorna a composição correta do IBOV naquele momento. Eliminar survivorship apenas no backtest mas treinar com universo viciado também invalida o sistema — por isso a correção é aplicada de ponta a ponta.

11. Fontes de dados — arquitetura em camadas

Nenhuma fonte sozinha cobre bem todas as necessidades do sistema. O JEMPO combina múltiplas fontes em camadas, com prioridades explícitas e deduplicação. Cada fonte externa é uma classe Python própria em `agents/sources/`, testável isolada e com tolerância a falha — se uma cair, as outras continuam alimentando o pipeline.

11.1 Notícias — múltiplas camadas com whitelist

Fonte	Peso	Cobertura	Limitação
Bloomberg CSV	1.00	Manual, eventos chave	ToS proíbe scraping automatizado
Reuters/Valor via GDELT	0.95	2015+	Geralmente só título e URL
GDELT geral	0.90	2015+	Volume alto, rate limit 1 req/5s
NewsAPI	0.85	Últimos 30 dias	Não cobre histórico

Whitelist rígida de domínios: `bloomberg.com`, `reuters.com`, `valor.globo.com`, `valor.com.br`, `broadcast.com.br`, `estadao.com.br`, `infomoney.com.br`. Qualquer fonte fora dessa lista é descartada — controle de qualidade na origem.

Bloomberg entra como camada de máxima qualidade via CSV manual. Não há API pública e os termos de uso proíbem scraping. A integração manual cobre ~80-100 eventos mais importantes do período (resultados grandes, fatos relevantes, decisões macro do Copom). Para volume e cobertura ampla, o GDELT preenche a lacuna sem custo de licenciamento — implementado em `agents/sources/gdelt.py` com throttling automático para respeitar o rate limit do GDELT.

Deduplicação fuzzy entre fontes: a mesma notícia frequentemente aparece em mais de uma fonte. O JOURNAL usa `difflib.SequenceMatcher` (`stdlib`) com threshold de 0.85 e janela temporal de 24h para identificar duplicatas. Processadas em ordem decrescente de peso, a primeira de cada grupo é sempre a da fonte mais confiável.

11.2 Preços — yfinance com duas versões

Preços vêm do `yfinance`, biblioteca aberta que consome dados da Yahoo Finance. Coletamos duas versões em paralelo: uma ajustada por split e dividendos (`Close`), padrão para análise de longo prazo, e outra bruta (`Close_raw`), necessária para cálculos de retorno de operação em estratégia de curto prazo (holding de 5 dias). Para holding curto, ajustar por dividendo distorce — o dividendo é distribuição de caixa que não se captura no intraday. Entregar as duas versões deixa a decisão final com o MATH&ML, sem o JOURNAL impor critério.

11.3 Fundamentos — CVM

Detalhes técnicos da migração para a CVM estão na seção 4. Resumindo aqui: fonte regulatória oficial brasileira, cobertura histórica completa desde 2011, `DT_RECEB` para anti-lookahead preciso, `TTM` para fluxo e point-in-time para balanço, tratamento BPB para bancos. `yfinance` permanece apenas como fallback para o campo "setor" (classificação setorial estável).

11.4 Macro — BCB SGS primário com FRED fallback

Dados macroeconômicos vêm do BCB SGS (Sistema Gerenciador de Séries Temporais do Banco Central do Brasil) — API pública e gratuita, sem necessidade de chave. Séries usadas: Selic diária anualizada (série 11), Selic meta (série 432), PTAX USD/BRL venda (série 1), IPCA mensal (série 433). IPCA acumulado 12 meses é composto multiplicativamente a partir do índice mensal. Como mencionado na seção 9.6, o IPCA é cortado pela data de divulgação (~11 dias após fim do mês), não pela referência, evitando vazamento de inflação ainda não pública.

FRED (Federal Reserve de St. Louis) é configurado como fallback automático via try/except. Se o BCB SGS ficar indisponível durante uma execução, o sistema busca dados equivalentes no FRED em vez de quebrar. Redundância operacional barata — o BCB é mais granular e atualizado, mas o fallback existe para resiliência em backtests longos.

12. Métricas e apresentação para a banca

12.1 Métricas reportadas

- Retorno acumulado da estratégia versus Ibovespa no período de backtest.
- Índice de Sharpe anualizado da estratégia, com retorno livre de risco igual à Selic média do período.
- Taxa de acerto (proporção de operações com retorno positivo) e payoff (razão ganho médio / perda média).
- Número total de operações executadas — sinal de robustez estatística.
- Maximum drawdown (queda máxima do pico ao vale) e tempo de recuperação.
- Distribuição de Monte Carlo: mediana, percentis 5 e 95, probabilidade de retorno positivo.
- Contribuição de retorno por setor e por categoria de notícia.

12.2 Três argumentos anti-crítica para a banca

Argumento contra lookahead bias:

Implementamos defesa em profundidade contra lookahead em três camadas. Timestamps timezone-aware em America/Sao_Paulo em todo o sistema, com corte de 17h05 da B3 e decisão na abertura para garantir que toda operação usa apenas dados até o fechamento de D-1. Para fundamentos, usamos o campo DT_RECEB da CVM — data exata de recebimento do documento — em vez de lag heurístico. Para o IPCA, cortamos pela data de divulgação (~11 dias após o mês de referência), não pela data de referência, evitando vazamento de inflação ainda não publicada. Validação defensiva em toda saída de dados levanta exceção se algum timestamp ultrapassar a data_limite. Mesmo bug futuro de fonte externa seria detectado antes de contaminar o backtest.

Argumento contra survivorship bias:

Mantemos UNIVERSO_HISTORICO com membership por data — cada ticker tem entrada e saída do IBOV. Em qualquer momento da simulação, apenas tickers ativos naquela data específica são considerados. Cobrimos 2019-2025 (warmup + calibração + treino + backtest). Incluímos casos emblemáticos como AMER3 (saída em 12/jan/2023 após pedido de RJ) e IRBR3 — o modelo treinou vendo quebras reais, não apenas empresas sobreviventes.

Argumento contra que o resultado foi sorte:

Rodamos 10.000 simulações de Monte Carlo a partir das operações realizadas, sorteando sequências alternativas com reposição. Em [X]% dos cenários a estratégia foi lucrativa e em [Y]% superou o Ibovespa. O resultado não depende de uma sequência específica improvável — é robusto. Adicionalmente, o backtest 2024-2025 é out-of-sample puro: o modelo nunca viu esses dados durante calibração ou treino, e o walk-forward retreina mensalmente apenas com dados realizados, sem espiar o futuro.

12.3 Perguntas prováveis e como responder

A banca tipicamente pergunta sobre dimensões específicas. Antecipamos as mais prováveis com respostas modeladas.

Como funciona o anti-lookahead?

Resposta resumida: três camadas — data_limite obrigatório nas funções, corte na fonte de cada API, validação defensiva pós-fetch. Timestamps timezone-aware. Corte 17h05 B3. DT_RECEB para fundamentos. IPCA cortado pela divulgação.

Por que GradientBoosting e não outros modelos?

Resposta resumida: LSTM precisa de muito mais dado; Random Forest é inferior em features correlacionadas. GradientBoosting com max_depth=3 e learning_rate=0.05 regulariza explicitamente para evitar overfitting num domínio com baixa razão sinal/ruído.

Como justificam essas features?

Baseadas em literatura acadêmica: momentum (Jegadeesh & Titman 1993), Post-Earnings Announcement Drift (Bernard & Thomas 1989), fatores de qualidade (Fama & French 1992) e contágio setorial. Não é busca livre por padrões.

Como tratam crises e regimes adversos?

Treino 2020-2023 inclui COVID, alta da Selic e queda de tech. Drawdown protection pausa entradas se queda do pico exceder 15%. Universo histórico inclui empresas que quebraram — modelo aprende com elas.

Quais são os custos de operação?

0.3% corretagem + 0.1% slippage por operação. Round-trip completo (entrada + saída) custa 0.8% sobre o capital alocado.

Por que a estratégia é robusta?

Monte Carlo confirma — não é uma sequência específica de eventos. Walk-forward adapta o modelo ao longo do tempo. Universo histórico elimina survivorship. Anti-lookahead garante que retornos são alcançáveis em operação real. Múltiplas fontes de notícia com tolerância a falha — se uma cair, o sistema continua.

13. Limitações conhecidas e próximos passos

Um projeto sério reconhece suas limitações — admitir fortalece a credibilidade. Listamos abaixo as limitações conhecidas do JEMPO e como as enquadramos perante a banca, junto com o caminho de evolução planejado.

13.1 Limitações honestas

- NewsAPI gratuito cobre apenas 30 dias. Compensado pela arquitetura em camadas com GDELTE e Bloomberg manual.
- Bloomberg sem API pública. Integração via CSV manual — viável para ~80-100 eventos mais relevantes do período, mas não escala para automação total.
- `publishedAt` das notícias é proxy do momento do evento — uma notícia das 14h pode analisar um fato das 10h. Documentado e assumido.
- Capital social da CVM tem gaps para empresas em reorganização (cisões, fusões). P/L e P/VP que dependem dessa peça são best-effort: retornam None com aviso, nunca número silenciosamente errado.
- B3 não disponibiliza API limpa para composição histórica do IBOV. Cobrimos os casos emblemáticos com confiança alta e marcamos TODOS onde resta verificação manual.
- GDELTE tem rate limit público de 1 requisição a cada 5 segundos por IP. Tratado via throttling automático no GDELTESource (sleep entre chamadas + retry com backoff), mas em backtests longos pode adicionar tempo de execução.
- Wayback Machine foi avaliada como camada adicional para notícias antigas, mas não foi implementada — as camadas atuais (Bloomberg + GDELTE + NewsAPI) já cobrem a janela necessária. Fica como melhoria futura se lacuna específica aparecer.
- Paginação das APIs GDELTE/NewsAPI não foi implementada (`maxrecords=100` por chamada). Suficiente para janelas de 7 dias por ticker, mas seria necessário para queries de janela longa.
- Walk-forward exige supervisão humana para detectar degradação do modelo em regimes muito novos. Em produção real, monitoramento contínuo de IC seria necessário.

13.2 Próximos passos imediatos

- Implementação dos agentes ECON, MATH&ML, ORQUESTADOR e PROGRAM. JOURNAL está concluído (mais de 70 testes passando).
- Coleta manual de notícias do Bloomberg para o período de backtest — checklist priorizada já produzida.
- Calibração do ECON em 2020-2021 com meta de IC > 0.15. Iteração de prompt limitada a 10 ciclos para evitar overfitting do prompt.
- Treino do MATH&ML em 2020-2023 com TimeSeriesSplit de 5 folds.
- Execução do backtest 2024-2025 com walk-forward mensal e Monte Carlo de 10.000 simulações.
- Preparação dos slides para a entrega de 31 de julho (pré-relatório) e 17 de agosto (entrega final).

13.3 Visão de evolução

Com recursos e tempo adicionais, o JEMPO evoluiria em três direções claras:

- Automação completa: integração com corretora via API (XP, BTG, Itaú) para execução real de ordens. O ORQUESTRADOR passaria de simulador para executor, com monitoramento 24/7 e alertas automáticos. Exige aprovação regulatória e auditoria de risco antes de qualquer produção.
- Expansão de mercados: aplicar a mesma arquitetura para FIIs (fundos imobiliários), BDRs (recibos de ações estrangeiras), ETFs setoriais. O ECON é agnóstico ao ativo — qualquer mercado com notícias e fundamentos é candidato.
- Publicação acadêmica: a correlação medida na calibração do ECON entre score de IA generativa e retornos reais é potencialmente publicável em journals de finanças quantitativas aplicadas. Transformaria o projeto de desafio universitário em contribuição metodológica.

13.4 Impacto potencial

O JEMPO demonstra que é possível construir uma estratégia quantitativa sofisticada combinando ferramentas abertas (Claude API, yfinance, scikit-learn, dados da CVM e BCB) com rigor metodológico. O custo de desenvolvimento é acessível — o diferencial é o conhecimento econômico, a disciplina metodológica e a integração entre fundamento qualitativo e ML quantitativo, não orçamento de infraestrutura.

Isso tem implicação prática: fundos menores, family offices e gestoras independentes brasileiras, que não têm acesso à infraestrutura de grandes casas, poderiam usar arquitetura similar para competir com mais eficiência no mercado. O JEMPO é, neste sentido, uma prova de conceito de democratização de finanças quantitativas no Brasil.

Sistema JEMPO — Desafio Quant AI 2026 — Itaú Asset Management